Getting Started

Data Structures and Algorithms, Imperial College Business School

Instructor: Prof. Reza Skandari, r.skandari@imperial.ac.uk

This document walks you through setting up for this module, step by step. You don't need any prior experience with coding, terminals, or any of the tools mentioned below; everything is explained as we go. You can either install everything on your own laptop, or use GitHub Codespaces to work in the cloud instead; see "Choose your setup" below.

By the end, you'll have:

- VS Code (the program we'll write and run code in) installed, or a GitHub Codespace set up as an alternative.
- Python (the programming language we'll use) installed, along with the packages and VS Code extensions this course needs (already done for you if using Codespaces).
- A basic idea of how to use a terminal, and how to run Python code in three different ways.

Contents

1. Choose your setup
2. GitHub Codespaces
3. Navigating and running Python
    - 3.1 Using the terminal
    - 3.2 Three ways to run Python
    - 3.3 Check everything works

Appendix: Installing locally (automatic installation, manual installation)

---

1. Choose your setup

You have two options for where you write and run your code this module. You can also switch between them week to week.

- **GitHub Codespaces (Section 2 below) is the setup we support and recommend**: a full VS Code environment that runs in your browser (or in the desktop VS Code app), with Python and everything else already installed. Nothing to install locally, and it works identically on every computer, so we can help fastest if anything goes wrong.
- **Local install** (see the Appendix below): Python and VS Code installed directly on your own laptop. We'll help as much as we can if something goes wrong, but see the note below.

> **A note on the install scripts.** These have only been tested on a handful of systems, so they may not run smoothly on yours — every laptop differs (existing software, admin rights, OS version), and we can't test every combination in advance. Feel free to try it. If it doesn't work, your best fallback is switching to **Codespaces**, not the manual installation steps in the Appendix — and either way, ask ICT or a TA for help rather than spending a long time troubleshooting alone. (The manual steps are genuinely good practice to learn, just better saved for later in the module, or if you already have some experience with this kind of setup.)

Whichever you pick, Section 3 (navigating and running Python) applies exactly the same way either way, so read that regardless.

---

2. GitHub Codespaces

A few terms used below, if you haven't used GitHub before: **GitHub** is a website for storing and sharing code online. An **organisation** on GitHub is just an account shared by a group (here, Imperial), which can own repositories only its members can see. A **repository** (or "repo") is one project's folder of files, hosted there. Once you open one in VS Code, it calls that open project folder your **workspace**, and its **file explorer** is the panel of files and folders down the left side of the window. An **extension** is a small add-on that gives VS Code an extra feature; the notebook support and the zip-creation tool you'll use below are both extensions.

1. **Register for GitHub through Imperial**, if you haven't already:
    1. Go to github.com/orgs/ImperialCollegeLondon/sso.
    2. Click **Continue**, and sign in with your Imperial credentials.
    3. When prompted to create a GitHub account, you can create a new one or use an existing one.

    4. Once signed in, you'll have access to Imperial's GitHub organisation, including the repository in the next step.
2. **Launch a Codespace**: make sure you've completed step 1 first, then go to github.com/Imperial-Business-School/dsa-2026-workspace and click **Code**, select the **Codespaces** tab, then click the **+** icon to create a codespace on `main`. You'll need to be signed in to GitHub to create a Codespace, even though the repository page itself is publicly viewable. The first launch can take up to 5 minutes while it installs everything; every launch after that is much faster. The first time, VS Code may ask **"Do you trust the authors of the files in this folder?"**; this is a normal one-time prompt for any new workspace, not a warning specific to this repository. Click **Yes, I trust the authors**. If **Compress** (an extension you'll use in step 3) doesn't appear when you right-click a file the very first time you open your Codespace, just reload the browser page and it'll show up; this is normal VS Code behaviour right after a fresh install, not a problem with the extension itself.
3. **Each week**, download that week's materials from Dropbox as usual, and unzip it. Depending on whether you downloaded a single class or several together, you may need to look inside a folder or two to find it, but find that week's **`activity` folder**. Select everything **inside** that `activity` folder (not the folder itself), and drag those files and folders into your Codespace's file explorer, so they land directly in your workspace rather than nested inside an extra folder. When you're done, click into the file explorer and press **Ctrl+A** (Windows/Linux) or **Cmd+A** (Mac) to select everything, then right-click and choose **Compress**. Rename the resulting zip to match the naming convention (e.g. `class-3-activity.zip`), same as the local workflow, then right-click that zip file and choose **Download**, and upload it to Canvas to submit. Once you've submitted, **confirm you actually have the downloaded zip saved locally** (check your Downloads folder), then delete those files (and the zip) from your Codespace, ready for next week's. Full step-by-step instructions are in this repository's README.

Note that Codespaces is only your coding environment; **you still submit your work via Canvas**, the same way and to the same place as everyone using a local install. There's no git-based submission in this module.

> **Always download your work before you close out for the day, not just before the final deadline.** Don't rely on the cloud as your only copy: a Codespace can be deleted (including automatically, after a period of inactivity), and GitHub's free tier only keeps unused Codespaces around for a limited time. If yours is deleted or something goes

> wrong with it, anything you didn't already download is gone. Downloading regularly, as described in step 3, protects you from this.

Codespaces needs no local installation at all. Once yours is set up, skip straight to **Section 3** below (navigating and running Python) — everything there applies inside a Codespace exactly the same way as on your own machine.

---

3. Navigating and running Python

This section applies whether you're using GitHub Codespaces (Section 2) or a local install (see the Appendix): the terminal and the ways of running Python work identically either way.

3.1 Using the terminal

We'll use the **terminal** (also called the command line) throughout this course. It's just a window where you type text commands instead of clicking on things: a different way of talking to your computer. Windows and Mac each have their own terminal programs (PowerShell and Terminal/bash, respectively) with mostly-overlapping commands. Learning your way around a terminal (whichever kind) is a genuinely useful, transferable skill well beyond this course; most software development and data work relies on it.

What a folder structure looks like

Before we get to commands, it helps to picture what we're navigating. Imagine you've downloaded this course's materials into a folder called `dsa`. Folders can contain other folders (and files) inside them, forming a structure like a tree with branches. It might look like this:

```
dsa/
├── class-1/
├── class-2/
├── class-3/
├── ...
├── class-10/
├── homeworks/
│   ├── hw-01/
│   └── hw-02/
└── setup/
    ├── install_windows.ps1
```

```
    ├── install_mac.sh
    └── getting-started.pdf
```

Each line is a folder or file. `dsa` is the top-level (or "parent") folder; `class-1`, `class-2`, etc. are folders inside it; and `setup` is another folder inside `dsa`, which itself contains files like `install_mac.sh`. The terminal commands below let you move around a structure like this and see what's inside each part.

Opening a terminal

- **Windows:** press the Windows key, type `PowerShell`, press Enter.
- **macOS:** press `Cmd+Space`, type `terminal`, press Enter.

You can also open a terminal from inside VS Code with `Ctrl+`` (backtick) or `Cmd+``.

Commands you'll use most often

Below, `$` represents the prompt (the terminal is waiting for you to type something). You don't type the `$` itself, just what comes after it. This is how the prompt looks on macOS. On Windows, PowerShell shows something like `PS C:\Users\yourname>` instead of `$`: same idea, it's just telling you where you currently are before waiting for your command.

The good news: as long as you're using **PowerShell** on Windows (which is what we've told you to open), every command below works exactly the same way, with the exact same name, on both Windows and Mac. We double-checked. (The one exception: if you ever end up in the older Windows **Command Prompt** instead of PowerShell, use `dir` instead of `ls`, and plain `cd` instead of `pwd`, to see your current folder.)

A general tip: terminals are unforgiving about typos: type commands exactly as shown, including spaces and capitalisation. `CD` is not the same as `cd`, and a missing space (e.g. `cdclass-1` instead of `cd class-1`) won't work. If something doesn't do what you expect, re-check exactly what you typed before assuming something is broken.

See what's in the current folder

```
$ ls
class-1  class-2  class-3  homeworks  setup
```

`ls` is short for "list".

Move into a folder

```
$ cd class-1
```

`cd` is short for "change directory" (a "directory" is another word for "folder"). After this, you're inside the `class-1` folder; any command you run next applies there.

Move back out, to the parent folder

```
$ cd ..
```

`..` always means "the folder one level up".

Create a new folder

```
$ mkdir class-1
```

`mkdir` is short for "make directory".

See the full path of where you currently are

```
$ pwd
/Users/yourname/dsa
```

`pwd` is short for "print working directory". This works the same in PowerShell as on Mac.

A short example, putting it all together:

```
$ cd Desktop
$ mkdir dsa
$ cd dsa
$ pwd
/Users/yourname/Desktop/dsa
$ ls
(nothing yet -- the folder is empty)
```

This creates a new empty folder called `dsa` on the Desktop, and moves into it.

We recommend keeping all of this course's materials together in one folder (e.g. `dsa`), so it's easy to find and to point VS Code at.

3.2 Three ways to run Python

There are three different ways you'll run Python code in this course. They look different, but underneath, it's always the same Python. It's worth understanding how they differ before you start.

1. The interactive interpreter (REPL)

REPL stands for **R**ead-**E**val-**P**rint **L**oop: Python **reads** what you typed, **evaluates** it (works out the result), **prints** that result, and then **loops** back to wait for your next line. In other words, it runs one line of code at a time and immediately shows you the answer: good for quick experiments or checking how something works.

Open a terminal (see 3.1 above) and type `python`, then press Enter (this works the same way on Windows and Mac):

```
$ python
Python 3.12.7 (main, ...) on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

The `>>>` is the Python prompt: Python is now waiting for you to type something. Try:

```
>>> print("Hello, world!")
Hello, world!
>>> 2 + 2
4
```

An important detail: each line you type **builds on the ones before it**, for as long as this session stays open. For example:

```
>>> x = 5
>>> x + 1
6
```

Here, `x` was created on the first line, and is still remembered on the second line. This is different from the other two ways of running Python below, where each run normally starts completely fresh.

When you're done, exit with:

```
>>> exit()
```

Notice the parentheses: `exit()`, not `exit`. This is worth pausing on, because it's a useful early lesson: in Python, `exit` on its own just refers to the exit function (without running it), while `exit()` actually **calls** it. The `()` is what tells Python "run this now." If you type just `exit` without parentheses, Python won't exit; it'll print something like `Use exit() or Ctrl-D (i.e. EOF) to exit`, reminding you to add the parentheses. You'll see this same pattern everywhere: `print` refers to the print function, `print()` actually prints.

2. Running a `.py` script (a file full of code)

A `.py` file ("Python file" or "script") holds a block of code you write out fully, then run all at once, as opposed to typing commands one at a time like in the interpreter above. Most of our class activities are `.py` files with functions you'll complete.

Creating, opening, and editing a new `.py` file, step by step:

1. Open VS Code.
2. Go to `File > New File...`. When it asks what kind of file, choose "Python File" (or just type a name ending in `.py`, e.g. `hello.py`). This opens a new, empty file in the editor.
3. Save it into your course folder: `File > Save As...`, name it `hello.py`, and save it somewhere easy to find (e.g. your `dsa` folder from 3.1 above).
4. Type the following into the file:

```
name = "Reza"
print("Hello, " + name)
```

1. Save the file (`Ctrl+S` / `Cmd+S`).

Running it:

- **From VS Code:** click the "Run" (▶) button in the top-right corner, or press `Ctrl+F5` / `Cmd+F5`. You should see `Hello, Reza` printed in a terminal panel that opens at the bottom.
- **From the terminal:** `$ python hello.py`

Try changing `"Reza"` to your own name, save again, and re-run. You should see the output change.

A script doesn't automatically show you the result of every line the way the interactive interpreter does: it only shows you what you explicitly ask it to show, using `print(...)`.

A useful VS Code trick: you don't always have to run the whole file. You can click and drag with your mouse to **highlight (select) just a few lines** of code; a small "Run" (▶) button will appear near the selection. Click it, and only the highlighted part runs, not the whole file. This is handy for testing one function without re-running everything above it.

For most class activities, you won't be creating a new `.py` file like this. Instead, you'll open a file we've already given you (e.g. `greedy.py`) and fill in the missing parts, then run it the same way.

3. Running a `.ipynb` notebook

A **notebook** mixes code, its output, and explanatory text together in one document, split into separate blocks called **cells**. This is what most of our class workbooks use, and is a common format in data analysis more generally.

Creating a new `.ipynb` file, step by step:

1. In VS Code, open the Command Palette (`Ctrl+Shift+P` / `Cmd+Shift+P`, a search box for commands, and one of the most useful things to know in VS Code).
2. Type `Create: New Jupyter Notebook` and press Enter. This opens a blank notebook with one empty cell.
3. Click into the cell and type:

```
print("Hello, world!")
```

Running it:

- Click into a code cell and press `Shift+Enter` to run just that cell and move to the next one (or `Ctrl+Enter` / `Cmd+Enter` to run it and stay where you are).
- The first time you run a cell, VS Code will ask you to **select a kernel** (the specific Python installation that should run your code): choose the Python environment you set up earlier (this may be called `dsa-env`, or something like `.venv`, depending on how you set it up, or "Python Environments" if you're using Codespaces).
- Output (text, plots, tables, etc.) appears directly underneath the cell that produced it.

For most class activities, you won't be creating a new notebook like this. Instead, you'll open a workbook `.ipynb` file we've already given you and fill in the missing parts, then run its cells the same way.

One thing to watch out for: unlike a `.py` script (which always runs top to bottom in order), notebook cells can be run **in any order**, and each one remembers whatever ran before it, similar to the interactive interpreter above. This is very convenient, but it also means it's easy to get confused: a variable might still hold a value from a cell you already edited or deleted. If a notebook starts behaving strangely, use "Restart Kernel and Run All Cells" (usually in a menu or toolbar button at the top) to get a clean run from scratch, top to bottom.

3.3 Check everything works

- Open VS Code, open this course's folder (`File > Open Folder...`).
- Open any class's `.ipynb` workbook file.
- Run the first code cell, as described in 3.2 above. VS Code may ask you to select a kernel the first time: choose the Python environment you set up earlier.
- If the cell runs and shows output underneath it, you're set up correctly.

If anything doesn't work, don't worry: note down the exact error message and ask a TA or Reza in class, or run the automatic setup script from the Appendix again, which produces a log file with the full details of what went wrong.

Appendix: Installing locally

See "A note on the install scripts" in Section 1 before starting: these scripts have only been tested on a handful of systems, so Codespaces (Section 2) is the more reliable choice if you're not confident troubleshooting a failed install yourself.

Automatic installation

We've written a script (a file that runs a series of commands automatically) that installs everything for you: VS Code, Python, and everything else listed above. This is the easiest way to get set up locally, if you'd rather not do it all by hand (see "Manual installation" below for that). Importantly, it installs the **exact Python version we use in class (3.12)**. If you ever install anything by hand instead, make sure to double-check you're getting this same version.

Step 1: Extract the setup files

If these setup files came to you as a single file ending in `.zip` (a compressed folder), you need to **extract** it before you can use anything inside; you can't run a script directly out of a zip file.

- **Windows:** find the `.zip` file (usually in your Downloads folder), right-click on it, and choose **"Extract All..."**. Choose where to put the extracted folder (or keep the default), and click **Extract**. This creates a normal folder with the same name, containing the actual files. Use that folder from now on, not the `.zip`.
- **macOS:** find the `.zip` file, and simply **double-click it**. This automatically creates a folder with the same name, next to the zip file, containing the extracted contents.

Once extracted, continue with the folder itself, not the `.zip`.

Step 2: Run the installer

On Windows

1. Find the folder you downloaded these setup files into (it should contain a file called `install_windows.ps1`, among others).
2. Right-click on `install_windows.ps1` and choose **"Run with PowerShell"**. (Or, if you'd rather: press the Windows key, type `PowerShell`, and press Enter to open a blue window yourself. Then get to the folder with the setup files either by typing `cd` followed by a space and **dragging the folder into the PowerShell window** (this types out the full path for you), or by typing the path yourself, e.g. `cd Downloads\setup` (see Section 3 above for more on moving between folders in a terminal). Press Enter, then type `.\install_windows.ps1` and press Enter. This does the same thing, just by typing the command instead of right-clicking.)
3. A window will open and start printing out what it's doing. This can take several minutes: installing Python and VS Code takes a while, especially on a slower internet connection.

If you see a red error message mentioning "running scripts is disabled on this system": this is Windows blocking script execution by default as a security measure, not a problem with the script itself. Fix it by opening PowerShell yourself (Windows key, type `PowerShell`, press Enter, **not** as Administrator), then running this once:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

Type `Y` and press Enter if asked to confirm. Then navigate to the setup folder as described above and run `.\install_windows.ps1` again.

If you see "'winget' was not found": your Windows installation doesn't have the App Installer package, which the script relies on. Install **App Installer** from the Microsoft Store (apps.microsoft.com/detail/9nblggh4nns1), then run the script again the same way as before.

On Mac

1. Find the folder you downloaded these setup files into (it should contain a file called `install_mac.sh`, among others).
2. Open a **Terminal** window: press `Cmd+Space` (hold the Command key and press Space), a search box will pop up, type `terminal`, and press Enter. A window with a text prompt will open; this is the Terminal.
3. Get to the setup folder. Either:
   - Type `cd` followed by a space (don't press Enter yet), then find the setup folder in Finder and **drag it into the Terminal window** (this automatically types out the full path to the folder for you). Now press Enter. Or:
   - Type the path yourself, e.g. `cd ~/Downloads/setup`, and press Enter.

   Either way, you should see the folder name change in the Terminal prompt, showing you've moved into it.
4. Type `bash install_mac.sh` and press Enter.
5. The Terminal will start printing out what it's doing. This can take several minutes.

If a separate macOS window pops up asking to install "Command Line Tools": this is expected on the very first run, on a Mac that's never had these tools before. Click **Install** in that window and let it finish (it can take several minutes on its own), then come back to Terminal and run the exact same command again: `bash install_mac.sh`. The script deliberately stops itself the first time so this installer can complete; running it a second time picks up right where it left off.

If Terminal asks for your password (this happens once, while installing Homebrew): type it and press Enter, even though **nothing appears on screen as you type**, not even dots. This is normal Terminal behaviour for password entry, not a sign it's frozen.

Step 3: Open VS Code

Now that everything's installed, here's how to open VS Code, since you'll need to do this a few times in this document:

- **Windows:** press the Windows key, type `Visual Studio Code`, and press Enter.
- **macOS:** press `Cmd+Space`, type `Visual Studio Code`, and press Enter (or find it in your Applications folder, or in Launchpad).

If something goes wrong

If you miss a step, or a command doesn't get typed exactly right, something will very likely not work: that's normal, and it's part of the learning experience, not a sign you've done something wrong as a person. **Don't worry if it breaks.** Just raise your hand and ask for help.

The script keeps a log of everything it did, including any step that failed, and tells you at the end exactly where that log is saved. **Don't try to troubleshoot this alone**: just ask a TA or Reza in class, and bring your laptop (with the log file still on it) so we can take a look together. This kind of setup issue is common and normal; it's not something you're expected to fix by yourself.

If you'd rather install everything by hand instead of using the script (or want to understand what it's doing), see **Manual installation** below, which walks through the same steps by hand.

Manual installation

These are the same steps the automatic script performs, done by hand: useful if the script doesn't work on your machine, or if you just want to understand what's being installed and why. It's genuinely good practice to work through this manually at some point, but if you're new to this kind of setup, **Codespaces is the easier and more reliable choice for right now**; come back to this later in the module once you're more comfortable, if you're curious.

1. Install VS Code

VS Code is the program (an "editor", or more precisely an "IDE", short for Integrated Development Environment) we'll use to write and run code throughout the module.

- Download the installer from code.visualstudio.com/download, choosing the option for your operating system (Windows or macOS).
- Run the installer, accepting the default options.

2. Install Python

We use a fixed Python version for the whole course: **Python 3.12**. This matters: **please install this exact version, not whatever version you might already have**. Using a different version can cause code that works for us in class to behave differently (or not work at all) on your machine, and makes it much harder for a TA to help you debug, since your setup won't match ours.

- **Windows:** download the Python 3.12 installer from python.org/downloads. During install, make sure to tick **"Add python.exe to PATH"** on the first screen before clicking Install; this makes Python easy to find later.
- **macOS:** the easiest route is Homebrew, a popular tool for installing software on a Mac via the terminal. If you don't already have it:
- Open Terminal and run the command shown on brew.sh (it starts with `/bin/bash -c "$(curl ...`). This may ask for your Mac password; that's normal.
- Once Homebrew is installed, run: `brew install python@3.12`

3. Create a virtual environment and install packages

A **virtual environment** is a self-contained copy of Python just for this course, kept separate from anything else on your computer, so the specific package versions we use here can't clash with anything else you might install later. The easiest way to create one is directly inside VS Code:

1. Open VS Code, then open the course folder you created in Section 3.1 (`File > Open Folder...`). The environment will be created inside whichever folder is currently open, so make sure this is done first.
2. Press `Ctrl+Shift+P` (Windows) or `Cmd+Shift+P` (Mac) to open the **Command Palette** (a search box for commands you'll use a lot).
3. Type `Python: Create Environment` and press Enter.
4. Choose **Venv** when asked (not Conda).
5. VS Code will list the Python installations it can find. **Make sure to select Python 3.12**, the version you just installed in step 2 above. If more than one version is listed, check the version number carefully next to each one; picking the wrong one is an easy mistake and means you won't be using the same setup as the rest of the class.
6. VS Code creates a new environment (usually in a folder called `.venv`) and automatically makes it your active Python interpreter for this folder; there's no separate "point VS Code at it" step needed after this.

7. Open a terminal inside VS Code (`Ctrl+`` / `Cmd+``); it should already show the new environment is active. Install the required packages by running: `python -m pip install numpy pandas matplotlib networkx xlrd ipykernel`

This installs numpy, pandas, matplotlib, networkx, xlrd, and ipykernel (the same list found in `config.json` in this folder, under `"packages"`). These are add-on libraries the course exercises rely on.

> **Prefer the terminal instead of VS Code's menus?** You can create the same environment by hand:
>
> - **Windows:** `py -3.12 -m venv %USERPROFILE%\dsa-env` then `%USERPROFILE%\dsa-env\Scripts\python.exe -m pip install numpy pandas matplotlib networkx xlrd ipykernel`
> - **macOS:** `python3.12 -m venv ~/dsa-env` then `~/dsa-env/bin/python -m pip install numpy pandas matplotlib networkx xlrd ipykernel`
>
> If you do it this way, you'll then need to manually tell VS Code to use it: press `Ctrl+Shift+P` / `Cmd+Shift+P`, type **"Python: Select Interpreter"**, and choose the interpreter at `%USERPROFILE%\dsa-env\Scripts\python.exe` (Windows) or `~/dsa-env/bin/python` (Mac). If it's not listed, choose "Enter interpreter path..." and paste the path in yourself.

4. Install VS Code extensions

Extensions are optional add-ons that give VS Code extra abilities. Open VS Code, go to the Extensions panel (`Ctrl+Shift+X` / `Cmd+Shift+X`), search for each of the following by name, and click **Install**:

Extension	Why we use it
Python (`ms-python.python`)	Core Python support and debugger
Pylance (`ms-python.vscode-pylance`)	Fast autocomplete and type-checking
Python Environments (`ms-python.vscode-python-envs`)	GUI panel to see/create/switch Python environments; helps with selecting the `dsa-env` venv

Extension	Why we use it
Jupyter (`ms-toolsai.jupyter`)	Run and edit `.ipynb` notebooks, used in most classes
Jupyter Keymap (`ms-toolsai.jupyter-keymap`)	Familiar Jupyter-style notebook keybindings
Jupyter Notebook Renderers (`ms-toolsai.jupyter-renderers`)	Richer output rendering (e.g. plots) in notebooks
HTML Preview (`george-alisson.html-preview-vscode`)	Preview the `.html` activity files we ship directly in VS Code
Rainbow CSV (`repreng.csv`)	Readable, coloured view of the `.csv` data files used in several classes
PDF (`tomoki1207.pdf`)	View PDFs (syllabus, assessment, etc.) without leaving VS Code
GitHub Pull Requests (`github.vscode-pull-request-github`)	GitHub account integration

(This exact list also lives in `config.json` in this folder, under `"extensions"`.)